

NOSQL: CIAEC49

Unit-II

Introduction and CRUD Operations in MongoDB

CRUD Operations

- **CRUD Operations** (Create, Read, Update, and Delete) are the basic set of operations that allow users to interact with the MongoDB server.
- As we know, to use MongoDB we need to interact with the MongoDB server to perform certain operations like entering new data into the application, updating data into the application, deleting data from the application, and reading the application data.

- MongoDB data types refer to the different types of data that can be stored in a MongoDB database. MongoDB supports a wide range of MongoDB data types, including strings, integers, doubles, boolean, dates, arrays, object IDs, regular expressions, and binary data.

Different Data Types

- **String**
- In MongoDB, the string data type is used to store a sequence of UTF-8 characters. Strings are one of the most commonly used MongoDB data types, as they can be used to represent a wide range of text-based data, such as **names, addresses, descriptions, and more.**
- Here's an example of how to use the string data type in MongoDB:
- Suppose we have a collection of documents representing customers, and each document has a field called "name" that contains the customer's name. We can define this field as a string data type, like this:

String

Source Code:

```
{  
  "_id": ObjectId("6156a98db6a0221ebf2c1ebc"),  
  "name": "John Smith",  
  "age": 35,  
}
```

Integer

Source Code:

```
{  
  "_id": ObjectId("6156a98db6a0221ebf2c1ebd"),  
  "name": "Widget",  
  "price": 10,  
  "description": "A basic widget for everyday use"  
}
```

- we have a collection of documents representing products, and each document has a field called "price" that contains the product's price in dollars. We can define this field as an integer data type

Boolean

Source Code:

```
{  
  "_id": ObjectId("6156a98db6a0221ebf2c1ebe"),  
  "name": "Gizmo",  
  "price": 20,  
  "in_stock": true,  
  "description": "A popular gizmo for tech enthusiasts"  
}
```

Min and Max Key

Source Code:

```
db.products.aggregate([  
  { $group: { _id: null, minPrice: { $min: "$price" } } }  
])
```

The Min/Max Keys data type in MongoDB is a special type used to compare a value against the lowest or highest possible value. This data type is used for finding minimum and maximum values in a collection, as well as for querying geospatial data.

Double

Source Code:

```
{  
  "_id": ObjectId("6156a98db6a0221ebf2c1ebe"),  
  "date": ISODate("2021-10-01T00:00:00Z"),  
  "amount": 1234.56,  
  "description": "Payment for services rendered"  
}
```

The Double data type in MongoDB is used to store floating-point numbers that require higher precision than the standard 32-bit float data type. Double data types are used to represent decimal values and are commonly used in financial applications or scientific calculations.

Array

Source Code:

```
{  
  "_id": ObjectId("6156b05cb6a0221ebf2c1ebf"),  
  "title": "10 Tips for Writing Better Code",  
  "body": "Lorem ipsum dolor sit amet...",  
  "tags": ["programming", "coding", "best practices"]  
}
```

Timestamp

Source Code:

```
{  
  "_id": ObjectId("6156b35eb6a0221ebf2c1ec0"),  
  "username": "johndoe",  
  "email": "johndoe@example.com",  
  "last_updated": Timestamp(1632354600, 1)  
}
```

MongoDB uses a BSON Timestamp internally in the oplog.

The Timestamp type works similarly to the Java Timestamp type. Use the Date type for operations involving dates.

```
Timestamp( { "t": <integer>, "i": <integer> } )
```

remove

To remove N number of documents in your collection myCollection:

Source Code:

```
db.getCollection('myCollection').find({}).limit(N).forEach(function(doc){  
    db.getCollection('myCollection').remove({_id: doc._id});  
})
```

Projection

The positional \$ operator is used in the projection document of the find() or findOne() method to limit the contents of an array to return the first element that matches the query condition on the array. This operator is particularly useful when you have an array of elements and you only need to retrieve the first element that matches your query criteria.

Here's an example of how to use the \$ operator in a projection:

```
db.collection.find(  
  { "arrayField": { $gte: 85 } },  
  { "arrayField.$": 1 }  
)
```

This query will return only the first element in the arrayField array that is greater than or equal to 85.

limit

You can't set a **limit** when using **remove** or **findAndModify**. So, if you want to precisely **limit** the number of documents removed,

Source Code:

```
db.collectionname.find().sort({"timestamp"-1}).limit(10)
```

Skip

- **MongoDB skip()**
- In [MongoDB](#), the **skip()** method will skip the first n [document](#) from the query result, we just need to pass the number of **records/documents** to be skipped. It allows us to control the starting point of the returned results, effectively removing the first n documents from the result set.
- **For example**, if our result has 5 records in it and we want to remove the first two records from it then we can use skip(2) at the end of the query.
- In other words, this method calls on a **cursor** to control from where MongoDB starts returning the results.

Ex:db.gfg.find().skip(1)

CRUD Operations

-

-

C —————> **Create**

R —————> **Read**

U —————> **Update**

D —————> **Delete**

CRUD Operations

- The **create or insert operations** are used to insert or add new documents in the collection. If a collection does not exist, then it will create a new collection in the database.
- You can perform, create operations using the following methods provided by the MongoDB:

Method	Description
<code>db.collection.insertOne()</code>	It is used to insert a single document in the collection.
<code>db.collection.insertMany()</code>	It is used to insert multiple documents in the collection.
<code>db.createCollection()</code>	It is used to create an empty collection.

CRUD Operations

```
> db.student.insertOne({  
... name : "Sumit",  
... age : 20,  
... branch : "CSE",  
... course : "C++ STL",  
... mode : "online",  
... paid : true,  
... amount : 1499  
... })  
{  
    "acknowledged" : true,  
    "insertedId" : ObjectId("5e540cdc92e6dfa3fc48ddae")  
}  
> █
```

CRUD Operations

```
anki — mongo — 80x55
> db.student.insertMany([
... {
...   name : "Sumit",
...   age : 20,
...   branch : "CSE",
...   course : "C++ STL",
...   mode : "online",
...   paid : true,
...   amount : 1499
... },
... {
...   name : "Rohit",
...   age : 21,
...   branch : "CSE",
...   course : "C++ STL",
...   mode : "online",
...   paid : true,
...   amount : 1499
... }
... ])
{
  "acknowledged" : true,
  "insertedIds" : [
    ObjectId("5e540d3192e6dfa3fc48ddaf"),
    ObjectId("5e540d3192e6dfa3fc48ddb0")
  ]
}
```

2. Read Operations

The Read operations are used to retrieve documents from the collection, or in other words, read operations are used to query a collection for a document.

You can perform read operation using the following method provided by the MongoDB:

Read Operations

Method	Description
db.collection.find()	It is used to retrieve documents from the collection

Read Operations

- **Read Operations Example**
- Let's look at some examples of Read operation from CRUD in MongoDB.
- **Example:** In this example, we are retrieving the details of students from the student collection using [db.collection.find\(\)](#) method.

CRUD Operations

```
|> db.student.find().pretty()  
{  
  "_id" : ObjectId("5e540cdc92e6dfa3fc48ddae"),  
  "name" : "Sumit",  
  "age" : 20,  
  "branch" : "CSE",  
  "course" : "C++ STL",  
  "mode" : "online",  
  "paid" : true,  
  "amount" : 1499  
}  
{  
  "_id" : ObjectId("5e540d3192e6dfa3fc48ddaf"),  
  "name" : "Sumit",  
  "age" : 20,  
  "branch" : "CSE",  
  "course" : "C++ STL",  
  "mode" : "online",  
  "paid" : true,  
  "amount" : 1499  
}  
{  
  "_id" : ObjectId("5e540d3192e6dfa3fc48ddb0"),  
  "name" : "Rohit",  
  "age" : 21,  
  "branch" : "CSE",  
  "course" : "C++ STL",  
  "mode" : "online",  
  "paid" : true,  
  "amount" : 1499  
}
```

3. Update Operations

Method	Description
db.collection.updateOne()	It is used to update a single document in the collection that satisfy the given criteria.
db.collection.updateMany()	It is used to update multiple documents in the collection that satisfy the given criteria.
db.collection.replaceOne()	It is used to replace single document in the collection that satisfy the given criteria.

3. Update Operations

```
> db.student.updateOne({name: "Sumit"},{$set:{age: 24 }})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 0 }
> db.student.find().pretty()
{
  "_id" : ObjectId("5e540cdc92e6dfa3fc48ddae"),
  "name" : "Sumit",
  "age" : 24,
  "branch" : "CSE",
  "course" : "C++ STL",
  "mode" : "online",
  "paid" : true,
  "amount" : 1499
}
{
  "_id" : ObjectId("5e540d3192e6dfa3fc48ddaf"),
  "name" : "Sumit",
  "age" : 20,
  "branch" : "CSE",
  "course" : "C++ STL",
  "mode" : "online",
  "paid" : true,
  "amount" : 1499
}
{
  "_id" : ObjectId("5e540d3192e6dfa3fc48ddb0"),
  "name" : "Rohit",
  "age" : 21,
  "branch" : "CSE",
  "course" : "C++ STL",
  "mode" : "online",
  "paid" : true,
```

3. Delete Operations

Method	Description
db.collection.deleteOne()	It is used to delete a single document from the collection that satisfy the given criteria.
db.collection.deleteMany()	It is used to delete multiple documents from the collection that satisfy the given criteria.

3. Delete Operations

```
,
> db.student.deleteOne({name: "Sumit"})
{ "acknowledged" : true, "deletedCount" : 1 }
> db.student.find().pretty()
{
  "_id" : ObjectId("5e540d3192e6dfa3fc48ddaf"),
  "name" : "Sumit",
  "age" : 20,
  "branch" : "CSE",
  "course" : "C++ STL",
  "mode" : "online",
  "paid" : true,
  "amount" : 1499,
```

3. Delete Operations

```
> db.student.find().pretty()  
{  
  "_id" : ObjectId("5e540d3192e6dfa3fc48ddaf"),  
  "name" : "Sumit",  
  "age" : 20,  
  "branch" : "CSE",  
  "course" : "C++ STL",  
  "mode" : "online",  
  "paid" : true,  
  "amount" : 1499,  
  "year" : 2020  
}  
{  
  "_id" : ObjectId("5e54103592e6dfa3fc48ddb1"),  
  "name" : "Rohit",  
  "age" : 21,  
  "branch" : "CSE",  
  "course" : "C++ STL",  
  "mode" : "online",  
  "paid" : true,  
  "amount" : 1499  
}  
> db.student.deleteMany({})  
{ "acknowledged" : true, "deletedCount" : 2 }  
> █
```

Comparison Query Operators

- **MongoDB – Comparison Query Operators**
- Comparison Query Operators in MongoDB are used to filter documents based on some specific criteria within their fields.
- MongoDB uses various comparison query operators to compare the values of the documents.

Query selectors

Query selectors are operators used in database queries to filter and retrieve specific documents based on certain conditions. They're essential for querying databases, such as MongoDB, by applying conditions to select and manipulate data.

Here's an overview of common types of query selectors.

- 1. Comparison Selectors**
- 2. Logical Selectors**
- 3. Element Selectors**
- 4. Evaluation Selectors**
- 5. Array Selectors**
- 6. Geospatial Selectors (for geospatial data)**
- 7. Bitwise Selectors**

List of comparison query operators:

Operators	Description
<u>\$eq</u>	Matches the values of the fields that are equal to a specified value.
<u>\$ne</u>	Matches all values of the field that are not equal to a specified value.
<u>\$gt</u>	Matches values of the fields that are greater than a specified value.
<u>\$gte</u>	Matches values of the fields that are greater than equal to the specified value.
<u>\$lt</u>	Matches values of the fields that are less than a specified value
<u>\$lte</u>	Matches values of the fields that are less than equal to the specified value
<u>\$in</u>	Matches any of the values specified in an array.
<u>\$nin</u>	Matches none of the values specified in an array.

1. \$eq (Equal)

- Matches documents where the field value is equal to the specified value.

// Find documents where age is exactly 25

```
db.collection.find({ "age": { "$eq": 25 } })
```

2. \$ne (Not Equal)

- Matches documents where the field value is not equal to the specified value.

// Find documents where age is not 25

```
db.collection.find({ "age": { "$ne": 25 } })
```

4. \$gt (Greater Than)

- Matches documents where the field value is greater than the specified value.

// Find documents where age is greater than 25

```
db.collection.find({ "age": { "$gt": 25 } })
```

5. \$lt (Less Than)

- Matches documents where the field value is less than the specified value.

// Find documents where age is less than 25

```
db.collection.find({ "age": { "$lt": 25 } })
```

6. \$lte (Less Than or Equal)

- Matches documents where the field value is less than or equal to the specified value.

// Find documents where age is 25 or less

```
db.collection.find({ "age": { "$lte": 25 } })
```

7. \$in (In)

- Matches documents where the field value is in the specified array of values.

// Find documents where status is either "A" or "B"

```
db.collection.find({ "status": { "$in": ["A", "B"] } })
```

7. \$gte (Greater Than or Equal)

- Matches documents where the field value is greater than or equal to the specified value.

// Find documents where age is 25 or more

```
db.collection.find({ "age": { "$gte": 25 } })
```

8. \$nin (Not In)

- Matches documents where the field value is not in the specified array of values.

// Find documents where status is neither "A" nor "B"

```
db.collection.find({ "status": { "$nin": ["A", "B"] } })
```

\$exists

9. \$exists

- Matches documents that contain (or do not contain) the specified field.

// Find documents where the field "name" exists

```
db.collection.find({ "name": { "$exists": true } })
```

// Find documents where the field "name" does not exist

```
db.collection.find({ "name": { "$exists": false } })
```

\$type

10. \$type

- Selects documents where the value of the field is an instance of the specified BSON type.

// Find documents where the age field is of type "int"

```
db.collection.find({ "age": { "$type": "int" } })
```

// Find documents where the name field is a string

```
db.collection.find({ "name": { "$type": "string" } })
```

\$regex

11. \$regex (Regular Expression)

- Matches documents where the field value matches the specified regular expression.

// Find documents where the name starts with "Jo"

```
db.collection.find({ "name": { "$regex": "^Jo" } })
```

// Find documents where the name contains "son"

```
db.collection.find({ "name": { "$regex": "son" } })
```

Logical Selectors

2. Logical Selectors

- These allow combining multiple query conditions.

Selector	Description	Example
\$and	Joins query clauses with a logical AND.	{ "\$and": [{ "status": "A" }, { "age": { "\$gt": 25 } }] }
\$or	Joins query clauses with a logical OR.	{ "\$or": [{ "status": "A" }, { "age": { "\$gt": 25 } }] }
\$not	Inverts the effect of a query expression.	{ "age": { "\$not": { "\$gt": 25 } } }
\$nor	Joins query clauses with a logical NOR (documents that do not match any conditions).	{ "\$nor": [{ "status": "A" }, { "age": { "\$gt": 25 } }] }

Geospatial Selectors

- **3. Geospatial Selectors**

Geospatial selectors are query operators specifically designed to work with geospatial data in databases like MongoDB. They enable you to search and analyze data based on geographic locations and spatial relationships, which is essential for applications involving mapping, geolocation, GIS, and spatial data analysis. Here's a look at commonly used geospatial selectors:

\$near

1. \$near

Purpose: Finds documents close to a specified point, sorted by distance.

Use case: Retrieving nearby locations, like finding the nearest stores, hospitals, or service centers to a user's current location.

```
{
  "location": {
    "$near": {
      "$geometry": {
        "type": "Point",
        "coordinates": [longitude, latitude]
      },
      "$maxDistance": distance_in_meters
    }
  }
}
```

Execution

```
mydatabase> db.mydatabase.createIndex({ location: "2dsphere" });
```

```
L location_2dsphere
```

```
mydatabase> db.mydatabase.createIndex({ location: "2dsphere" });
```

```
location_2dsphere
```

```
mydatabase> db.mydatabase.insertMany([
```

```
... {
...   location: {
...     type: "Point",
...     coordinates: [77.5946, 12.9716] // Example coordinates for Bangalore
...   }
... },
... {
...   location: {
...     type: "Point",
...     coordinates: [78.4867, 17.3850] // Example coordinates for Hyderabad
...   }
... }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6736d9ecb7f003bd13c73bfa'),
    '1': ObjectId('6736d9ecb7f003bd13c73bfb')
  }
}
```

Execution

```
my database> db.mydatabase.find({
  ... location: {
  ...   $near: {
  ...     $geometry: {
  ...   type: "Point",
  ...   coordinates: [77.5946, 12.9716] // Query location
  ...   },
  ...   $maxDistance: 10000 // 10 km radius
  ... }
  ... }
  ... });
```

Output:

```
[
  {
    _id: ObjectId('6736d9bfb7f003bd13c73bf8'),
    location: { type: 'Point', coordinates: [ 77.5946, 12.9716 ] }
  },
  {
    _id: ObjectId('6736d9ecb7f003bd13c73bfa'),
    location: { type: 'Point', coordinates: [ 77.5946, 12.9716 ] }
  }
]
```

2. \$nearSphere

- Purpose: Similar to \$near, but performs calculations on a spherical surface, which is more accurate for Earth-like round coordinates.
- Use case: Essential for distance calculations that take Earth's curvature into account, such as precise location-based services.
- Syntax Example:

```
{  
  "location": {  
    "$nearSphere": {  
      "$geometry": {  
        "type": "Point",  
        "coordinates": [longitude, latitude] },  
      "$maxDistance": distance_in_meters } } }
```

\$geoWithin

- **Purpose:** Finds documents within a specified geographic area, such as within a polygon, circle, or other shapes.
- **Use case:** Useful for defining service areas, city boundaries, or environmental regions, and finding locations within them.

- **Syntax Example**

```
{ "location": {  
  "$geoWithin": {  
    "$geometry": {  
      "type": "Polygon",  
      "coordinates": [ [ [lng1, lat1], [lng2, lat2], [lng3, lat3], [lng1, lat1] ] ] } } }
```

\$geometry

- Purpose: Specifies the shape and coordinates of a geospatial area or point to use in conjunction with other geospatial selectors like \$near, \$geoWithin, and \$geoIntersects.
- Use case: Defines the shape or point of interest in spatial queries.
- { "location": {
- "\$near": { "\$geometry": {
- "type": "Point",
- "coordinates": [longitude, latitude] } } } }

6. Geospatial Data Types & 7. Geospatial Indexing

- Point: Represents a single geographic coordinate, typically used for locations like a store or event location.
- LineString: Defines a path made up of multiple points, useful for routes or paths.
- Polygon: Represents an enclosed area, useful for boundaries or regions, such as city borders.
- 7. Geospatial Indexing
- 2D Index: Used for data that can be represented on a flat, 2D plane. Typically used when precise spherical calculations aren't necessary.
- 2dsphere Index: Ideal for geographic coordinates (latitude and longitude) on a spherical surface, such as Earth, and required for most MongoDB geospatial queries.

1. Bitwise Selectors

- 1. Bitwise Selectors
- Bitwise selectors are operators that allow for bitwise comparisons between a field and a specified number. They're useful when working with flags or binary values, often used to represent multiple Boolean values in a single field.
- `$bitsAllSet`: Matches documents where all specified bit positions are set to 1.
- Example: `{ "permissions": { "$bitsAllSet": 5 } }` matches documents where the binary representation of permissions has all the bits set as in binary 101 (5 in decimal).
- `$bitsAnySet`: Matches documents where at least one of the specified bit positions is set to 1.
- Example: `{ "statusFlags": { "$bitsAnySet": 2 } }` matches documents with at least one bit set as in binary 010.
- `$bitsAllClear`: Matches documents where all specified bit positions are set to 0.
- Example: `{ "permissions": { "$bitsAllClear": 4 } }` finds documents where the field permissions has the third bit (binary 100) set to 0.
- `$bitsAnyClear`: Matches documents where at least one of the specified bit positions is set to 0.
- Example: `{ "statusFlags": { "$bitsAnyClear": 3 } }` matches documents with at least one bit cleared in the first or second position (binary 011).

Execution

```
mydatabase> db.mydatabase.insertMany([
... { "permissions": 5 }, // binary 101
... { "permissions": 7 }, // binary 111
... { "permissions": 4 } // binary 100 (this won't match the query)
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6736dc31b7f003bd13c73bfc'),
    '1': ObjectId('6736dc31b7f003bd13c73bfd'),
    '2': ObjectId('6736dc31b7f003bd13c73bfe')
  }
}
mydatabase> db.mydatabase.find({ "permissions": { "$bitsAllSet": 5 } });
[
  { _id: ObjectId('6736dc31b7f003bd13c73bfc'), permissions: 5 },
  { _id: ObjectId('6736dc31b7f003bd13c73bfd'), permissions: 7 }
]
```

2. Sort Method

- 2. Sort Method
- The sort method in MongoDB arranges documents in a specified order. It's used to control the sequence in which documents appear in the result set.
- Syntax: `db.collection.find(query).sort({ field: 1 or -1 })`
- 1 for ascending order.
- -1 for descending order
- Example:
- `db.users.find().sort({ "age": 1, "name": -1 })`

3. Aggregation Functions

- Aggregation functions are used to perform calculations on data, often within the **aggregation pipeline**. Common aggregation functions include:
- **\$sum**: Calculates the total sum of numeric values.
 - **Example**
- `db.sales.aggregate([ {`
- `"$group": { "_id":`
- `"$productId",`
- `"totalSales": { "$sum": "$amount" } } ])`

\$avg

- Calculates the average of numeric values
- `db.sales.aggregate([ { "$group": { "_id": "$productId", "averageSales": { "$avg": "$amount" } } } ])`
- This finds the average sales amount for each product.

\$min and \$max:

- Find the minimum and maximum values, respectively
- `db.scores.aggregate([ {`
- `"$group": { "_id":`
- `"$studentId", "minScore": {`
- `"$min": "$score" },`
- `"maxScore": { "$max": "$score" } } ])`
- This finds the minimum and maximum score for each student.

\$count

Counts the number of documents that match the query.

- `db.users.aggregate([ {`
- `"$match": { "status": "active" } },`
- `{`
- `"$count": "activeUsers" } ])`
- Counts active users.

\$first and \$last:

- Retrieve the first or last value in a group, based on the specified order.
- `db.logs.aggregate([ {`
- `"$sort": { "timestamp": 1 } }, {`
- `"$group": { "_id": "$userId",`
- `"firstLogin": { "$first": "$timestamp" }, "lastLogin": { "$last":`
- `"$timestamp" } } ])`
- This finds the first and last login timestamps for each user.

Pattern Matching

- Pattern matching is used to match documents based on regular expressions, making it useful for finding documents with partially matching or complex string patterns.
- `$regex`: Performs pattern matching on strings using regular expressions.
- Syntax: `{ "field": { "$regex": /pattern/, "$options": "i" } }`
- `i` for case-insensitive matching.
- `m` for multiline matching.
- `db.users.find({ "username": { "$regex": /^john/i } })`
- This finds users whose usernames start with "john" (case-insensitive).

Example query Combining Bitwise, Sort, and Aggregation

- `db.events.aggregate([ {`
- `"$match": { "statusFlags": {`
- `"$bitsAnySet": 2 } } ]`, // Bitwise selector {
- `"$sort": { "timestamp": -1 } }]`, // Sort by latest event { "\$group": { "_id":
- `"$eventType",`
- `"totalCount": { "$count": {} }, "lastTimestamp": { "$max": "$timestamp"`
- `}} } ])`
- This query finds events with specific bitwise flags, sorts them by timestamp in descending order, and groups by eventType to calculate the total count and the latest event timestamp for each type.

